Software Requirements Specification

Use Case Based Capstone Template

Document field	Team entry
Project name	Savings Management
Team name and members	Stack Underflow (Zane, Jeff, Saw, Jaron)
Sponsor and instructor	Commerce Bank mentors and Syed Jawad Hussain Shah
Version and status	0.1 Draft
Date	10/2/2026
Requirements lead	[Enter owner]

How to use this template

This specification contains functional and nonfunctional requirements. Use cases are the primary method for gathering and organizing functional requirements. Nonfunctional requirements define measurable qualities and constraints that apply to the use cases or the product as a whole. Complete both categories before deriving sprint stories, tests, and release decisions. Text inside brackets [ ] tells you what to write. Replace it with your own information before submitting the document.

- Use stable identifiers such as UC-AUTH-01, BR-001, NFR-SEC-001, US-AUTH-03, and TC-AUTH-004.
- Write use cases from the actor's goal and the system's observable response. Avoid screen layout and database design unless they are true constraints.
- Use FR identifiers for separate functional statements and NFR identifiers for measurable quality or constraint statements.
- Keep the requirements document as the behavioral source. Link sprint stories and test cases back to a use case flow.
- Record approved changes in the revision history and update traceability during every sprint.

Revision history

Version	Date	Author	Change and approval
0.1	10/2/2026	Zane/Saw/Jeff/Jaron	Initial actor list, use case diagram, and catalog

Contents

1 Introduction

2 Stakeholders Actors and Goals

3 Product Context and Constraints

4 Use Case Model

5 Functional Requirements and Detailed Use Cases

6 Nonfunctional Requirements

7 Data and External Interfaces

8 Traceability and Sprint Mapping

9 Review and Change Control

Appendix A Reusable Detailed Use Case Form

1 Introduction

1.1 Purpose and audience

[State what this document specifies, who should review it, and how it will support design, sprint planning, testing, and sponsor review.]

1.2 Problem and product vision

Most people find it difficult to allocate money for saving purposes while still managing their everyday buying expenses. It becomes hard to track how one is progressing towards his or her financial goals when there is no set plan of savings, and there is no definite goal being worked towards. There may not be much awareness on the part of the user as to how close he or she is to achieving the savings goal.

This application works as an online financial management tool to help people save their money to reach financial goals. This app will receive data from the user about their spending and give feedback on how to improve their savings. These savings will be used to contribute to an emergency fund with a user set goal (ex: $1000). Then once the wishlist goal has been reached, the user can then create their own savings wishlists and put money away for these wishlists (ex: garage,gaming,gifts,etc). Wish lists will notify the user when savings has reached an amount that is equal to or greater than an item on the wishlist. History of transactions, savings, goals, etc will be viewable by the user.

1.3 Scope

In scope	Out of scope
Login & Register page, home page & dashboard, User profile & settings, AI element	[Explicit exclusions and deferred capabilities] Real web server deployment, Real User data in the database
Emergency Savings page, Wishlist page, User account balance, Financial tracking (User provided), Financial Feedback	Real banking connections, real money movement, and real card data unless the sponsor explicitly changes the brief

1.4 Definitions and conventions

Term	Definition
Actor	A person, role, device, or external system outside the product boundary that interacts with the product.
Use case	A goal-oriented interaction between an actor and the system.
Scenario	One path through a use case, including the main success path or an alternate or exception path.
Functional requirement	A capability, rule, validation, calculation, or system response that the product must provide.
Nonfunctional requirement	A measurable quality attribute or constraint on how the product operates or is built.
Must	Mandatory requirement.
Should	Desired requirement that may be deferred with an approved reason.

Term	Definition
Emergency Savings	A specified dollar amount that the user wants to achieve through saving. Specifically saved for the case of emergency events (Unexpected car repairs, medical, etc)
Wishlist	A list of items with an associated dollar amount that the user wants to save money to buy. Does not require purchase when the dollar amount of an item is achieved.
[Domain term]	[Project-specific definition]

1.5 Assumptions dependencies and open questions

ID	Type	Statement	Owner	Target resolution
A-001	Assumption	[Condition outside the team's control]	[Name]	[Date or sprint]
D-001	Dependency	[External service, decision, data, or environment]	[Name]	[Date or sprint]
Q-001	Open question	[Question requiring sponsor or instructor decision]	[Name]	[Date or sprint]

2 Stakeholders Actors and Goals

Identify people and external systems before naming use cases. An actor represents a role, not a specific teammate or screen.

2.1 Stakeholders

Stakeholder	Interest or decision	How requirements will be validated
Commerce Bank mentors	Sponsor alignment and technical feedback	Midsemester review, questions, and final walkthrough
Instructor	Course expectations and evaluation	Scheduled reviews and submission feedback
[Stakeholder]	[Interest or decision authority]	[Interview, review, observation, or demo]

2.2 Actors and user characteristics

Actor ID	Actor or role	Goals	Knowledge permissions and needs
ACT-01	Registered user	Track emergency savings progress, track wishlist savings progress	[Skill level, permissions, accessibility or usability needs] Skill level: medium Permissions: access to their personal budgeting data through the various pages which expose and visualize that, ability to change details about their own account
ACT-02	[Supporting actor or external system]	[Goals or service provided]	[Constraints and trust assumptions]
ACT-03	Unregistered User		

2.3 Actor goal summary

Actor	Goal	Success outcome	Candidate use case
Registered user	Create an emergency savings goal	User specified savings goal is saved to their account and viewable by the user	UC-ESAV-01
Registered user	Add money to account balance	User can specify how much they would like to add to their account balance, account balance is updated while adhering to ACID transaction requirements	UC-BAL-01
Registered user	Create a wishlist	User can create a wishlist with a name when the emergency savings goal has been achieved. Wishlist is viewable by the user and can be accessed.	UC-WSH-01
[Actor]	[Goal stated with a verb]	[Observable outcome]	[UC-ID]

3 Product Context and Constraints

3.1 Product environment and system boundary

[Describe the browser, front end, server or API, database, authentication, external services, and environments at the level needed to understand requirements. Reference the architecture document for design detail.]

Browser: any browser which accepts pages which use html, css, and javascript

Frontend: built using Javascript React with Bootstrap

Backend stack:

Web API framework:

OO layer:

Language: Java

Database:

3.2 Sponsor brief baseline

- Build a full-stack web application selected by the team; do not use a CMS such as WordPress.
- Include a meaningful financial component such as budget tracking, simulated payments, or investments.
- Support user registration and login; store passwords as secure hashes, never as plain text.
- Use a database, a front-end framework or library, and one CSS framework.
- Provide Login/Register, Home/Dashboard, User Profile/Settings, and at least two additional pages.
- Use only fake or dummy card and financial data when simulating transactions.
- Include unit testing; automated tests are strongly encouraged.
- Choose and complete at least two stretch goals from the sponsor brief.

3.3 Chosen stretch goals

Stretch goal	Reason for selection	Use cases or requirements affected	Evidence
Deploy the project into a windows server/cloud instance	Accessibility to physical servers makes this goal highly feasible.	[UC or NFR IDs]	Physical demonstration of application running on server
Create web api for communicating with frontend and backend	The value this offers is in more flexible communication methods between server and client. It is a feasible stretch goal.	[UC or NFR IDs]	Demonstration of communication between frontend and backend through the internet and a well structured api

3.4 Business rules

Rule ID	Rule	Source	Related use cases
BR-001	Development, testing, and demonstrations must use fabricated financial and card data.	Sponsor brief	[UC IDs]
BR-002	[Project rule independent of interface design]	[Sponsor, stakeholder, or team decision]	[UC IDs]

4 Use Case Model

4.1 Use case diagram

Insert a readable diagram that shows actors outside the system boundary and goal-oriented use cases inside it. Use include or extend only when the relationship improves understanding.

[Insert current use case diagram here]

4.2 Use case catalog

ID	Use case name	Primary actor	Priority	Status	Target	Owner
UC-AUTH-01	Register an account	Prospective user	Must	Draft	Sprint __	[Name]
UC-AUTH-02	Sign in	Registered user	Must	Draft	Sprint __	[Name]
UC-BUD-01	Create a monthly budget	Registered user	Must	Draft	Sprint __	[Name]
UC-___	[Goal-oriented name]	[Actor]	Must/Should /Could	Proposed/Draft /Approved	[Sprint]	[Name]

4.3 Use case relationships and scope notes

Relationship or boundary	Explanation and reason
[UC-A includes UC-B]	[Explain reusable required behavior; remove this row if include is not needed]
[UC-X extends UC-Y]	[Explain optional or conditional behavior; remove this row if extend is not needed]

Relationship or boundary	Explanation and reason
Out-of-scope actor goal	[Record a goal discussed with stakeholders that will not be implemented and why]

5 Functional Requirements and Detailed Use Cases

Use cases are the primary source of functional requirements. Copy the reusable form in Appendix A for every high-priority use case. Complete the main success scenario and realistic alternate and exception flows before the related story enters a sprint. Add a separate FR statement when behavior applies across use cases or needs more precision than a flow step provides.

5.1 UC ID Use case name

Field	Team entry
Use case ID and name	[UC-AREA-## Goal-oriented verb phrase]
Goal	[Value the primary actor wants]
Primary actor	[Actor ID and role]
Supporting actors	[External roles or systems, or None]
Trigger	[Event that starts this use case]
Preconditions	[Facts that must already be true]
Success postconditions	[System state and actor outcome after success]
Minimum guarantee	[System state preserved if the use case fails]
Priority status target owner	[Must/Should/Could; Draft/Approved; sprint or release; owner]

Main success scenario

Step	Actor or system	Action or response
1	Actor	[Actor action]
2	System	[Observable system response]
3	Actor	[Actor action]
4	System	[Validation, state change, and confirmation]

Alternate and exception flows

Flow ID	Related step	Condition	System behavior and recovery
3a	3	[Expected alternate condition]	[Behavior, message, state, and return point]
3b	3	[Invalid input or authorization failure]	[Behavior and recovery without unsafe partial data]
4a	4	[Dependency or database failure]	[Error handling, logging if selected, and retry path]

Rules data quality and evidence

Item	Linked IDs or details
Business rules	[BR IDs]
Data inputs and outputs	[Entities, fields, validation, fabricated test values]
Functional requirements	[FR IDs derived from or related to this use case]
Nonfunctional requirements	[NFR IDs relevant to this use case]
Acceptance and test evidence	[TC IDs, Given/When/Then examples, automated-test links]
Open questions and assumptions	[Q or A IDs]
Backlog mapping	[Story or task IDs and the specific scenario slice each implements]

5.2 Functional requirements catalog

Use this catalog for required behavior that applies across use cases or needs a precise standalone statement. Sponsor-brief capabilities such as required pages belong here, not in the nonfunctional section. Every functional requirement must link to at least one use case, business rule, or sponsor constraint and must be testable.

FR ID	Functional requirement	Source or related use cases	Priority	Verification
FR-AUTH-001	The system must allow a prospective user to create an account using the approved registration fields.	UC-AUTH-01	Must	TC-AUTH-001
FR-BUD-003	The system must reject a budget amount that is zero or negative and must not create a partial record.	UC-BUD-01 flow 3a	Must	TC-BUD-006
FR-___	[One required capability or behavior using must]	[UC, BR, or brief ID]	[Priority]	[Test ID or method]

6 Nonfunctional Requirements

State measurable quality attributes and constraints that apply across the product or to named use cases. Every NFR must include operating conditions, a target or threshold, and a verification method. Do not write vague statements such as the system must be fast or secure.

ID	Category	Requirement and measure	Verification	Priority
NFR-SEC-001	Security	Passwords must be stored only as salted hashes produced by the selected authentication library; plain-text passwords must not appear in storage or logs.	Integration inspection and log review	Must
NFR-PRIV-001	Privacy	The application and all demonstrations must use fabricated financial and card data and must not request real banking credentials.	Repository database and demo review	Must
NFR-USE-001	Usability	[Representative user task, success rate, time, and conditions]	[Usability test]	[Priority]
NFR-PERF-001	Performance	[Operation, dataset or load, response target, and percentile]	[Measured test]	[Priority]
NFR-ACC-001	Accessibility	[Keyboard, focus, label, contrast, or audit target]	[Audit or manual test]	[Priority]
NFR-REL-001	Reliability	[Failure behavior, recovery, or availability target]	[Test or observation]	[Priority]
NFR-MNT-001	Maintainability	[Test, review, modularity, or configuration expectation]	[Review or pipeline evidence]	[Priority]

6.1 Additional interface quality requirements

NFR ID	Quality requirement and measure	Related use cases	Verification
NFR-UI-001	For the selected representative workflows, all interactive controls must be reachable and operable by keyboard.	[UC IDs]	Keyboard walkthrough
NFR-UI-002	Labels, validation messages, and navigation must be understandable to representative users completing the selected tasks.	[UC IDs]	Usability test with stated success criterion
NFR-UI-___	[Measurable interface quality or accessibility constraint]	[UC IDs]	[Method and threshold]

7 Data and External Interfaces

7.1 Data requirements

Data ID	Entity or data	Important fields and validation	Ownership privacy and retention	Related use cases
DATA-USER	User account	[Fields and validation; password stored only as a hash]	[Owner, sensitivity, retention, deletion]	[UC IDs]
DATA-___	[Entity]	[Fields, types, ranges, uniqueness]	[Rules and prohibited data]	[UC IDs]

7.2 External interfaces

Interface ID	System or service	Purpose and contract	Authentication data and failure behavior	Owner
INT-001	[API, reporting tool, email service, or browser]	[Inputs, outputs, protocol, version]	[Trust, timeout, retry, unavailable behavior]	[Name]

8 Traceability and Sprint Mapping

8.1 Requirements traceability matrix

Use case or requirement	Scenario or statement	Story task	Design component	Test case	Status
UC-BUD-01	Main flow steps 1-5	US-BUD-04	Budget form API service and repository	TC-BUD-001	[Status]
UC-BUD-01	Alternate flow 3a	US-BUD-05	Validation rules and accessible message	TC-BUD-006	[Status]
[ID]	[Flow or statement]	[Issue ID]	[Component]	[TC ID]	[Status]

8.2 Sprint backlog mapping

Story or task	Use case slice	Acceptance criteria	Target sprint	Owner	Result
US-BUD-04	UC-BUD-01 main success scenario	[Observable result]	Sprint __	[Name]	Planned
[Issue ID]	[UC and flow IDs]	[Given/When/Then or concise checks]	[Sprint]	[Name]	[Status]

8.3 Prioritization

Explain how Must, Should, and Could priorities affect the MVP and sprint order. Mandatory sponsor requirements must be completed before optional stretch work. Record architecture or risk-driven sequencing when it differs from business priority.

9 Review and Change Control

9.1 Review checklist

[ ] Every actor has at least one goal and every in-scope goal maps to a use case.

[ ] Each detailed use case has a trigger, preconditions, main success scenario, alternate or exception flows, and postconditions.

[ ] The use case describes behavior and outcomes without unnecessary screen or database design.

[ ] Functional requirements describe testable behavior and use stable FR identifiers when standalone statements are needed.

[ ] Nonfunctional security, privacy, usability, performance, accessibility, and reliability requirements include measurable targets and verification methods.

[ ] Use cases, FRs, and NFRs link to sprint stories and test cases by stable identifier.

[ ] All examples, fixtures, screenshots, and demonstrations use fabricated financial data.

9.2 Change requests and decisions

Change ID	Requested change and reason	Affected IDs	Decision and approver	Date sprint
CHG-001	[Request and business or technical reason]	[UC BR NFR DATA IDs]	[Approved deferred rejected and by whom]	[Date / sprint]

Appendix A Reusable Detailed Use Case Form

Copy this entire appendix section for each use case. Replace every bracketed prompt and delete rows that do not apply only after the team confirms they are unnecessary.

Field	Team entry
Use case ID and name	[UC-AREA-## Goal-oriented verb phrase]
Goal	[Actor value]
Primary actor	[Actor ID and role]
Supporting actors	[Roles or external systems]
Trigger	[Starting event]
Preconditions	[Required prior state]
Success postconditions	[State after success]
Minimum guarantee	[State preserved after failure]
Priority status target owner	[Entries]

Main success scenario

Step	Actor or system	Action or response
1	[Actor/System]	[Action or observable response]
2	[Actor/System]	[Action or observable response]
3	[Actor/System]	[Action or observable response]
4	[Actor/System]	[Action or observable response]
5	[Actor/System]	[Action or observable response]
6	[Actor/System]	[Action or observable response]

Alternate and exception flows

Flow ID	Related step	Condition	System behavior and recovery
__a	__	[Condition]	[Behavior and return point]
__b	__	[Condition]	[Behavior and return point]
__c	__	[Condition]	[Behavior and return point]

Rules data quality and evidence

Item	Linked IDs or details
Business rules	[BR IDs]

Item	Linked IDs or details
Data inputs and outputs	[Data IDs and validation]
Functional requirements	[FR IDs]
Nonfunctional requirements	[NFR IDs]
Acceptance and test evidence	[TC IDs and acceptance examples]
Open questions and assumptions	[Q and A IDs]
Backlog mapping	[Story or task IDs with scenario slices]

Use case review record

Review field	Entry
Owner and last updated	[Name and date]
Reviewers	[Requirements reviewer, developer, tester, sponsor or instructor when applicable]
Status and target	[Proposed / Draft / Approved; target sprint or release]
Decision or follow-up	[Approved, changes requested, deferred question, and linked issue IDs]

[ ] The primary actor and goal are clear.

[ ] Success, alternate, and failure behavior are testable.

[ ] Business rules, functional requirements, nonfunctional requirements, sprint work, and tests are linked by ID.

[ ] The use case is ready for the target sprint only after review issues are resolved.